МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО «СЕВАСТОПОЛЬСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Факультет информационных технологий
ВТШ «СПИ»

ОТЧЕТ

о выполнении лабораторной работы № 10

Тема: «Библиотека SciPy»

по дисциплине

«Программирование на языке Python»

Вариант № 4

Выполнил:
Студент группы ИИ/б-25-6-о
Заварзин А. В.
Проверил:
Старший преподаватель кафедры ИТиС
Кузнецов С. А.

Севастополь, 2026

ЦЕЛЬ РАБОТЫ

Изучить библиотеку SciPy языка Python, научиться применять методы библиотеки SciPy при решении уравнений, интегрировании, работе с изображениями и в других практических задачах.

ПОСТАНОВКА ЗАДАЧИ И ВАРИАНТ ЗАДАНИЯ

Задание №1: Решение уравнений. Модуль optimize. Отделить корень графически (построить график функции) и уточнить его методом деления отрезка пополам (method='bisect'), используя библиотеку SciPy. Вывести корень, значение функции в корне и количество итераций.

Для варианта №4 дана следующая функция:

$$2x^3 - 3x^2 - 12x + 8 = 0. \quad x \in [2, 4]$$

Задание №2: Аппроксимация. Методом наименьших квадратов построить аппроксимирующую параболу вида $y_m = c_0 + c_1 x + c_2 x^2$ для функции, заданной в табличном виде. Вывести коэффициенты (c_0, c_1, c_2), сумму квадратов отклонений и построить график с исходными точками и кривой.

X: [0, 2, 4, 6]

Y: [-2.0, 1.0, 5.0, 10.2]

Задание №3: Интерполяция. Модуль SciPy.interpolate. Функция задана в виде таблицы. Вычислить значение функции в заданной точке x^*. Решить задачу линейной и квадратичной (или кубической) интерполяции с помощью функции interp1d(). Вывести значения и построить общий график (исходные точки, линейная и сплайновая интерполяция). Задание для варианта №4 приведено в таблице 10.1.

Таблица 10.1 – Условие задания №3 для варианта №4

Интерполяция	x	y	Найти
Линейная	[2, 4, 5]	[4, 16, 25]	f(3.5)
Квадратичная	[2, 4, 5]	[4, 16, 25]	f(3.5)

ХОД РАБОТЫ

Перед началом работы необходимо отметить, что все задания выполнены в интерактивной среде Jupyter.

Задание 1. Ниже приведён листинг кода для решения поставленной задачи.

```
def f(x):
    """
    Функция по варианту
    """
    return 2*x**3 - 3*x**2 - 12*x + 8

a, b = (-5, 5)
x = np.linspace(a, b, 400)
y = f(x)

plt.plot(x, y, label=r"$f(x)=2x^3-3x^2-12x+8$")
plt.axhline(0, color="black", linewidth=0.8)
plt.axvline(a, color="gray", linestyle="--", linewidth=0.8)
plt.axvline(b, color="gray", linestyle="--", linewidth=0.8)
plt.scatter([a, b], [f(a), f(b)], color="red", zorder=5)
plt.title(f"Отделение корня на отрезке [{a}, {b}]")
plt.xlabel("x")
plt.ylabel("f(x)")
plt.legend()
plt.grid(True, alpha=0.3)

res = scopt.root_scalar(f, bracket=[a, b], method="bisect")
root = res.root
f_root = f(root)
iters = res.iterations

plt.scatter([root], [f_root], color="green", zorder=6,
            label=f"корень ≈ {root:.6f}")
plt.legend()
plt.show()
```

```
print(f"Корень: x = {root:.10f}")
print(f"Значение функции в корне: f(x) = {f_root:.3e}")
print(f"Количество итераций: {iters}")
```

Реализованная программа рисует график на заданном отрезке, после чего с помощью библиотеки SciPy, а именно модуля SciPy.optimize уточняет корень функции методом деления отрезка пополам (с помощью функции root_scalar, в параметр method передаём значение “bisect”). Также в программе реализован подсчёт количества итераций для нахождения корня функции. Результаты работы программы приведены на рисунках 10.1 и 10.2.

```
Корень: x = -2.1519448174
Значение функции в корне: f(x) = -2.974e-11
Количество итераций: 43
```

Рисунок 10.1 – Результат работы программы

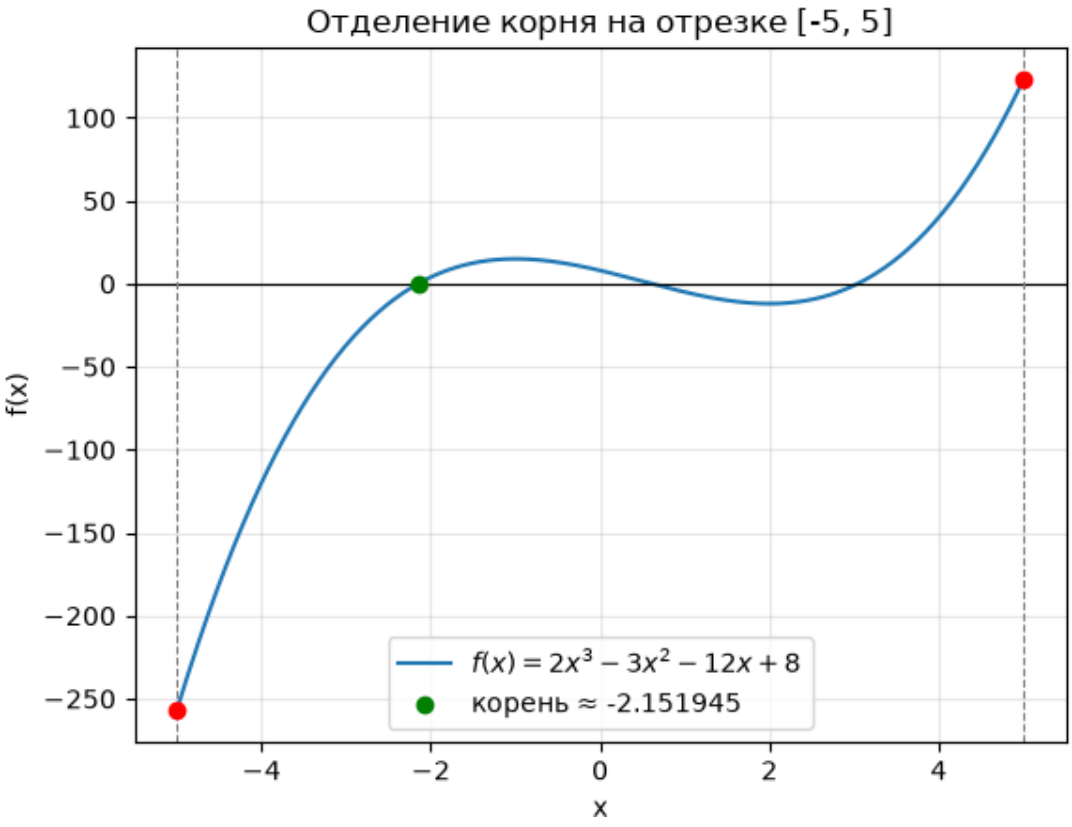


Рисунок 10.2 – График, построенный программой

Задание 2. Ниже представлен листинг кода для решения задачи.

```
# Исходные данные
x_data = np.array([0, 2, 4, 6])
y_data = np.array([-2.0, 1.0, 5.0, 10.2])

# Определяем вид аппроксимирующей функции
def parabola(x, c0, c1, c2):
    return c0 + c1 * x + c2 * x**2

# Находим коэффициенты методом наименьших квадратов
# popt - массив с оптимальными параметрами [c0, c1, c2]
# pcov - ковариационная матрица (нам сейчас не нужна, поэтому ставим _)
popt, _ = scopt.curve_fit(parabola, x_data, y_data)
c0, c1, c2 = popt

# Вычисляем предсказанные значения и сумму квадратов отклонений (SSE)
y_pred = parabola(x_data, *popt)
sse = np.sum((y_data - y_pred)**2)

# Вывод результатов
print("Коэффициенты аппроксимирующей параболы:")
print(f"c0 = {c0:.4f}")
print(f"c1 = {c1:.4f}")
print(f"c2 = {c2:.4f}")
print(f"\nУравнение: y = {c0:.4f} + {c1:.4f}*x + {c2:.4f}*x^2")
print(f"Сумма квадратов отклонений: {sse:.4f}")

# Построение графика
x_smooth = np.linspace(min(x_data) - 1, max(x_data) + 1, 100)
y_smooth = parabola(x_smooth, *popt)

# Исходные точки
plt.scatter(x_data, y_data, color='red', s=50, zorder=5, label='Исходные
точки')
# Аппроксимирующая кривая
plt.plot(x_smooth, y_smooth, color='blue',
         label=f'$y = {c0:.2f} + {c1:.2f}x + {c2:.2f}x^2$')
```

```
plt.title("Аппроксимация методом наименьших квадратов")
plt.xlabel("x")
plt.ylabel("y")
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()
```

Данная программа выполняет поиск коэффициентов оптимаьных параметров (c_0, c_1, c_2) с помощью функции curve_fit, параметрами функции передаём заданную функцию, а также массивы с заданными x и y. После чего мы на основе предсказанных значений оптимальных параметров вычисляем строим функцию на заданном промежутке и считаем сумму квадратов отклонений. После всех вычислений строим график с исходными точками и функциям, а также выводим результаты вычислений на экран. Результаты работы программы представлены на рисунках 10.3 и 10.4.

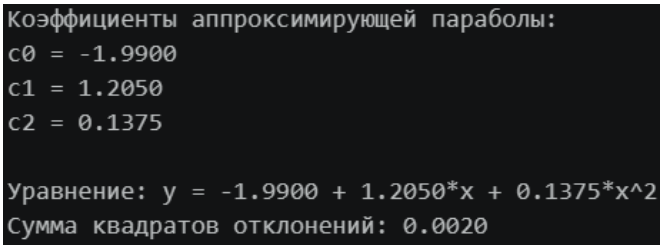


Рисунок 10.3 – Результат работы программы

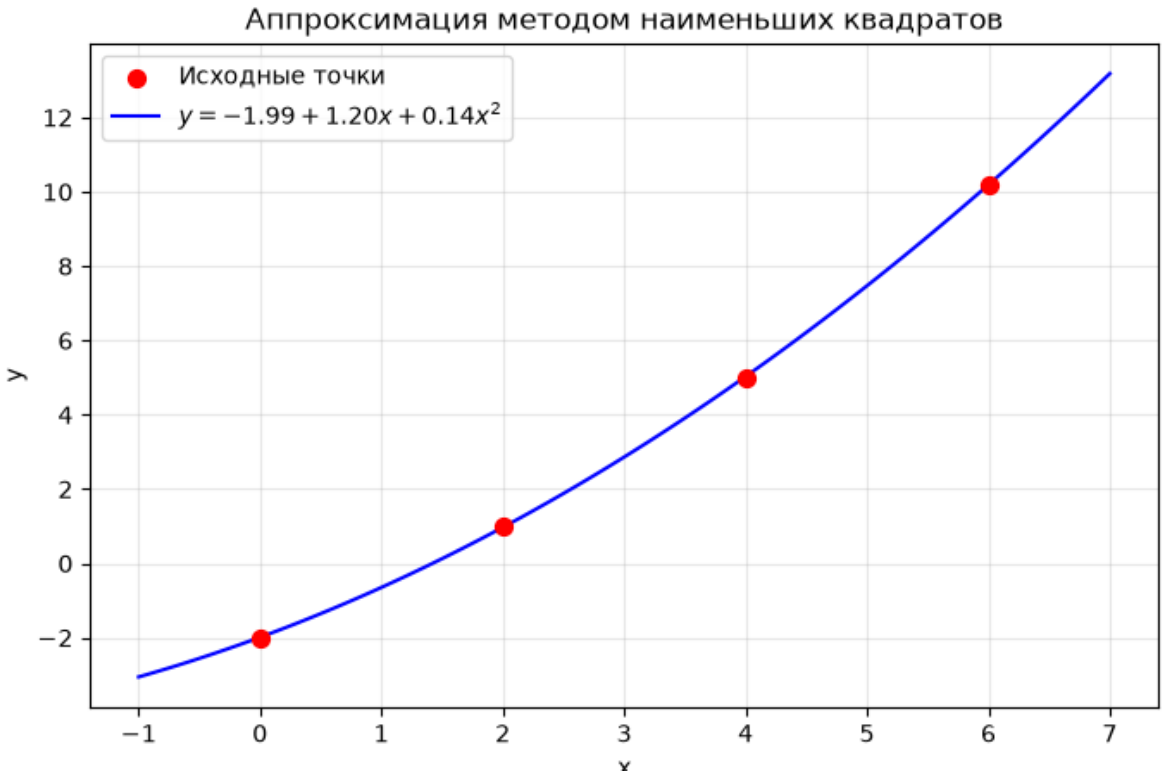


Рисунок 10.4 – График, построенный программой

Задание 3. Ниже приведён листинг кода для решения поставленной задачи.

```
import scipy.interpolate as scip

# Исходные данные
x = np.array([2, 4, 5])
y = np.array([4, 16, 25])
x_target = 3.5

# Линейная интерполяция
f_linear = scip.interp1d(x, y, kind='linear')
y_linear = f_linear(x_target)

# Квадратичная интерполяция
f_quad = scip.interp1d(x, y, kind='quadratic')
y_quad = f_quad(x_target)

# Вывод значений
print(f"Значение функции в точке x* = {x_target}:")
```

```
print(f"Линейная интерполяция:     f({x_target}) = {y_linear:.4f}")
print(f"Квадратичная интерполяция: f({x_target}) = {y_quad:.4f}")

# Построение графика
x_plot = np.linspace(2, 5, 100)

plt.figure(figsize=(8, 5))

# Исходные точки
plt.scatter(x, y, color='red', s=60, zorder=5, label='Исходные точки')

# Графики интерполяции
plt.plot(x_plot, f_linear(x_plot), '--', color='blue', label='Линейная
интерполяция')
plt.plot(x_plot, f_quad(x_plot), '-', color='green', label='Квадратичная
интерполяция')

# Отмечаем искомую точку на графике
plt.scatter(x_target, y_linear, color='blue', marker='X', s=100, zorder=6,
label='x*')
plt.scatter(x_target, y_quad, color='green', marker='X', s=100, zorder=6,
label='x*')

plt.title("Интерполяция функции")
plt.xlabel("x")
plt.ylabel("y")
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()
```

Данная программа производит линейную и квадратичную интерполяцию с помощью функции interp1d из модуля SciPy.interpolate. В качестве аргумента kind мы передаём вид интерполяции, в нашем случае ‘linear’ и ‘quadratic’ для линейной и квадратичной интерполяции соответственно. После чего выводим значения на экран и строим график с исходными точками и самими графиками интерполяций. Результаты работы программы приведены на рисунках 10.5 и 10.6.

```
Значение функции в точке x* = 3.5:
Линейная интерполяция:     f(3.5) = 13.0000
Квадратичная интерполяция: f(3.5) = 12.2500
```

Рисунок 10.5 – Результат работы программы

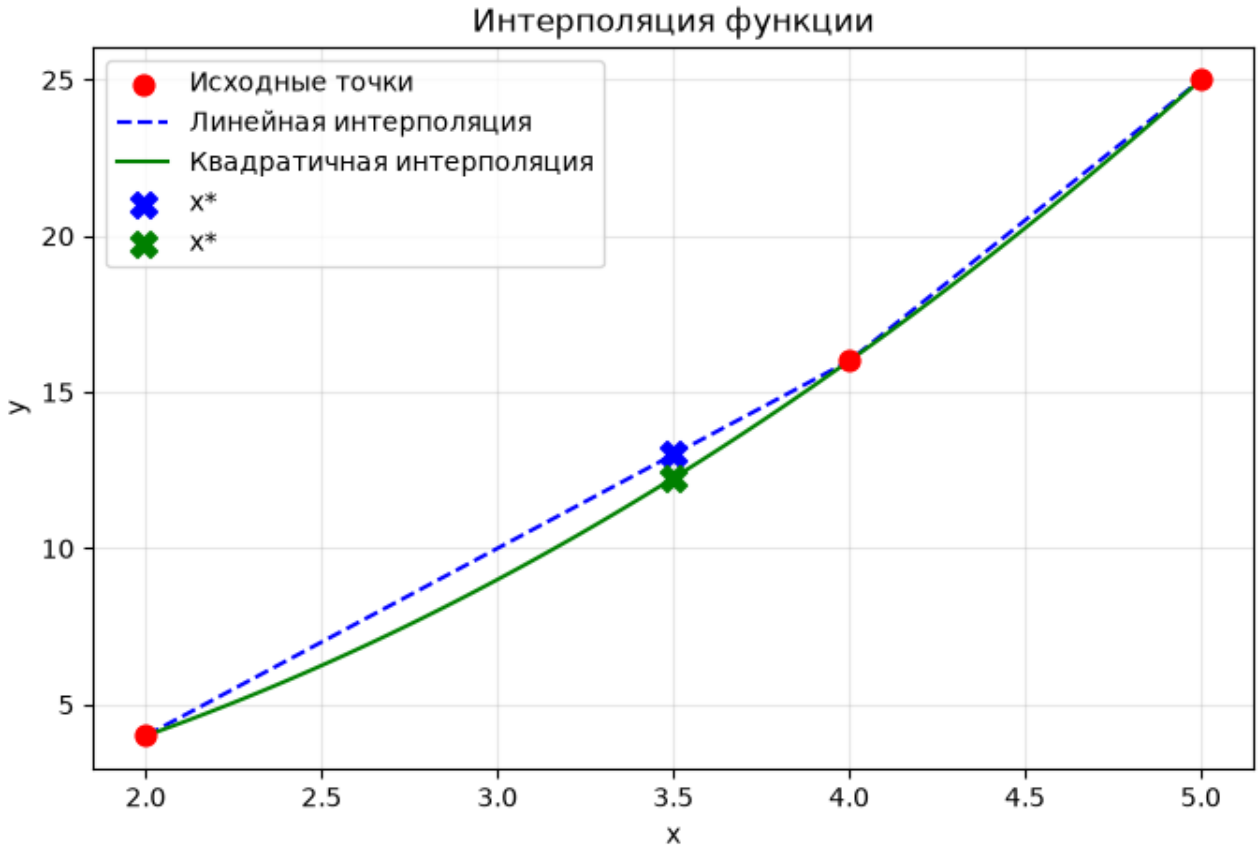


Рисунок 10.6 – График, построенный программой

ВЫВОДЫ ПО РАБОТЕ

В ходе выполнения работы были приобретены практические навыки работы с библиотекой для научных математических вычислений SciPy для языка программирования Python. В рамках лабораторной работы освоены методы решения нелинейных уравнений с помощью модуля optimize (функция root_scalar), аппроксимация экспериментальных данных методом наименьших квадратов с использованием curve_fit, а также линейная и квадратичная интерполяция на основе interp1d из модуля interpolate. Полученные результаты подтверждают высокую точность и удобство применения SciPy для решения типовых задач вычислительной математики.